Focus — usare git sul proprio lavoro

> **Materiale facoltativo.** Non è materia d'esame e non serve per nessun esercizio del corso. Per scaricare e aggiornare il materiale bastano i due comandi di `guida_GitHub.md`: questa scheda è per chi vuole usare git sulle **proprie** analisi.
>
> Vale la pena leggerla prima della tesi, non dopo.

Il problema che risolve

Ti è già capitato, o ti capiterà:

```
analisi.py
analisi_v2.py
analisi_v2_definitiva.py
analisi_v2_definitiva_CORRETTA.py
analisi_v2_definitiva_CORRETTA_questa_sì.py
```

Tre domande a cui quella cartella non sa rispondere:

- **cosa** è cambiato fra una versione e l'altra?
- **quale** ha prodotto i numeri che sono finiti nella relazione?
- si può tornare alla versione di **due settimane fa**?

Un sistema di controllo di versione registra la storia di un progetto file per file, modifica per modifica. Una copia sola, e tutta la storia dentro.

Senza	Con git
tante copie dello stesso file	una copia, e la storia completa
nessuna traccia di cosa è cambiato e perché	ogni modifica ha una descrizione
recuperare dipende dalla fortuna	qualunque stato passato è recuperabile

Per uno statistico il punto è il secondo: **la storia dei comandi e degli script è il verbale di cosa hai fatto ai dati.** È la stessa ragione per cui in `L02_shell.md` si preferisce la riga di comando al mouse, portata al livello del progetto.

I tre luoghi di git

```
cartella di lavoro        area di staging         repository
─────────────────         ───────────────         ──────────
 i tuoi file come          i file scelti           istantanee
 li stai scrivendo   -->   per la prossima   -->   permanenti
               git add     istantanea    git commit
```

Termine	Significato
repository	una cartella di cui git tiene la storia
commit	un'istantanea, con un nome e una descrizione
area di staging	dove scegli *cosa* entra nella prossima istantanea

L'area di staging sembra un'inutile complicazione e non lo è: serve a fare istantanee **sensate**. Se hai sistemato la lettura dei dati e nel frattempo hai lasciato a metà un grafico, metti in staging solo la prima e la registri da sola. Un commit che fa una cosa si capisce a distanza di mesi; un commit che ne fa cinque no.

Configurazione, una volta per macchina

```
$ git config --global user.name  "Nome Cognome"
$ git config --global user.email "nome.cognome@campus.unimib.it"
$ git config --global init.defaultBranch main
$ git config --list                # per controllare
```

Nome e mail finiscono in ogni commit: servono a dire chi ha fatto cosa.

Iniziare

```
$ mkdir ~/analisi_meteo && cd ~/analisi_meteo
$ git init
Initialized empty Git repository in /home/tuo.nome/analisi_meteo/.git/
```

La cartella `.git` nascosta è la storia: se la cancelli, resta la cartella e sparisce il passato. (Si vede con `ls -a` — il punto davanti al nome, come nella caccia al tesoro.)

Il ciclo di tutti i giorni

Sono tre comandi, e li ripeterai per sempre in quest'ordine.

```
$ git status                    # cosa è cambiato?
$ git add lettura.py            # scelgo cosa registrare
$ git commit -m "Leggo il meteo e sistemo i tipi delle colonne"
```

`git status` è quello che si usa più spesso: dice quali file sono nuovi, quali modificati e quali già in staging. Chiamalo ogni volta che non sei sicuro di dove sei — è il `pwd` di git.

Un messaggio di commit utile dice **perché**, non cosa: `"Sistemo il separatore e l'encoding del file ISTAT"` vale dieci volte `"modifiche"`.

Guardare la storia

```
$ git log --oneline              # una riga per commit
01d237a Aggiungo il grafico delle temperature mensili
a71e238 Leggo il meteo e sistemo i tipi delle colonne

$ git log --oneline -5           # solo gli ultimi cinque
$ git show a71e238               # cosa conteneva quel commit
```

E per vedere cosa hai cambiato **da allora**:

```
$ git diff                       # modifiche non ancora in staging
$ git diff --staged              # modifiche in staging, pronte al commit
```

`git diff` mostra le righe togliendo e aggiungendo: `-` quelle che c'erano, `+` quelle nuove. Su uno script è il modo più rapido di rispondere a «ma cosa ho toccato ieri sera?».

Tornare indietro

```
$ git restore analisi.py         # butta le modifiche non ancora in staging
$ git restore --staged analisi.py  # togli dallo staging, ma tieni le modifiche
```

La prima **non è recuperabile**: quelle modifiche non erano in nessuna istantanea, quindi git non le ha da nessuna parte. Tutto ciò che è stato committato, invece, si recupera sempre.

Cosa versionare, e cosa no

Ed è qui che il focus diventa specifico per chi lavora con i dati.

Sì:

- gli script e i notebook dell'analisi;
- le note e la documentazione;
- i file di configurazione.

No:

- **i dati grezzi.** Sono grandi, non cambiano, e non sono tuoi: un repository git conserva ogni versione di ogni file per sempre, quindi un CSV da 200 MB scaricato tre volte diventano 600 MB di storia inutile. I dati grezzi stanno in `data/raw` in sola lettura (`chmod 444`), e nel repository ci va lo **script che li scarica**, non il file;
- **gli output generati**: PDF, grafici, tabelle. Si rifanno lanciando il codice, ed è il senso di avere il codice;
- **le credenziali**: password, token, chiavi. Una volta committate restano nella storia anche se le cancelli dal file.

Le esclusioni si dichiarano in un file `.gitignore` nella radice del progetto:

```
data/raw/
out/
*.pdf
```

```
__pycache__/
.env
```

La regola in una riga: **versiona ciò che hai scritto, non ciò che puoi rigenerare o riscaricare.**

Mettere il proprio lavoro su GitHub

Per *leggere* un repository pubblico non serve nulla; per pubblicarne uno tuo serve un account, perché GitHub deve sapere chi scrive. Dopo averlo creato e aver creato il repository vuoto dal sito:

```
$ git remote add origin https://github.com/tuo-utente/analisi-meteo.git
$ git push -u origin main        # la prima volta
$ git push                       # tutte le volte dopo
```

Al primo `push` GitHub chiede di autenticarti, e **non accetta la password** dell'account: serve un *personal access token* (dal sito, *Settings* → *Developer settings* → *Personal access tokens*) oppure una chiave SSH. La procedura cambia ogni tanto: quella buona è sulla documentazione di GitHub, e per il nostro corso non ti serve.

Da un'altra macchina, poi:

```
$ git clone https://github.com/tuo-utente/analisi-meteo.git
$ git pull                       # per allinearti a quello che hai fatto altrove
```

Cosa resta fuori da questa scheda

Git ha molto altro — **branch**, **pull request**, **merge**, **fork** — e serve quando si lavora in più persone sullo stesso codice. Per un'analisi che scrivi tu, il ciclo `status`, `add`, `commit` più `push` copre tutto, ed è meglio padroneggiare quello che conoscere vagamente il resto.

Comando	Cosa fa
`git init`	inizia a tenere la storia di una cartella
`git status`	cosa è cambiato, cosa è in staging
`git add <file>`	metti nella prossima istantanea
`git commit -m "messaggio"`	registra l'istantanea, con il perché
`git log --oneline`	la storia, un commit per riga
`git diff` · `git diff --staged`	cosa è cambiato, riga per riga
`git restore <file>`	butta le modifiche non in staging (**definitivo**)
`git restore --staged <file>`	togli dallo staging, tieni le modifiche
`git remote add origin <url>`	collega il repository a GitHub
`git push` · `git pull`	manda · prendi